

overly terse code. Take the time to come up with descriptive, specific names for your variables and functions. A blank line in between sections of your code can serve the same function as paragraph breaks in a book, letting the reader know which parts are meant to be read together. This aphorism is roughly the same as beautiful is better than ugly.

Special cases aren't special enough to break the rules. Although practicality beats purity. These two aphorisms contradict each other. Programming is full of “best practices” that programmers should strive for in their code. Skirting these practices for a quick hack might be tempting but can lead to a rat's nest of inconsistent, unreadable code. On the other hand, bending over backward to adhere to rules can result in highly abstract, unreadable code. For example, the Java programming language's attempt to fit all code to its object-oriented paradigm often results in lots of boilerplate code for even the smallest program. Walking the line between these two aphorisms becomes easier with experience. In time, you'll learn not only the rules but also when to break them.

Errors should never pass silently. Unless explicitly silenced. Just because programmers often ignore error messages doesn't mean the program should stop emitting them. Silent errors can happen when functions return error codes or `None` instead of raising exceptions. These two aphorisms tell us that it's better for a program to fail fast and crash than to silence the error and continue running. The bugs that inevitably happen later on will be harder to debug because they're detected long after the original cause. Although you can always decide to explicitly ignore the errors your programs cause, be sure you're making the conscious choice to do so.

In the face of ambiguity, refuse the temptation to guess. Computers have made humans superstitious: to exorcise the demons in our computers, we perform the sacred ritual of turning them off and then on. Supposedly this will fix any mysterious problem. But computers are not magic. If your code isn't working, there's a reason why, and only careful, critical thinking will solve the problem. Refuse the temptation to blindly try solutions until something seems to work; often, you've merely masked the problem rather than solved it.